



NETWORKS AND SYSTEMS MANAGEMENT

Alexandre Cunha
up202400086

Pedro Januário
up202108768

MOTIVATION

- As part of our LGP (Laboratório de Gestão de Projetos) class, we're simulating a startup that delivers a mobile-first solution for journalists to **share video reports on the go**.
- The app supports **mobile uploads** (recorded or ingested content), backed by a service architecture that includes:
 - A backend application
 - The tusd resumable upload server
 - All traffic managed by a nginx reverse proxy

MOTIVATION CONT.

- To ensure a **production-ready and scalable environment**, we identified key needs:
 - **Automated deployment** of new backend versions with minimal human intervention
 - **Dynamic nginx configuration updates** without downtime
 - **Robust monitoring** for observability and service health tracking
 - This setup not only serves our current simulated startup needs but also lays the groundwork for **container orchestration with Kubernetes**
-

PROBLEMS

Human Error: Manual deployment and configuration increases the risk of mistakes, such as incorrect environment settings or misconfigured services.

Deployment Inconsistency: Without automation, it's hard to guarantee that each deployment is identical, leading to unpredictable bugs and behavior.

Downtime During Updates: Manual changes to services like nginx can cause temporary service outages or downtime during reloads.

Compatibility Issues: Integrating new services (e.g., tUSD) without structured workflows may introduce version conflicts or misalignments.

MORE PROBLEMS

Lack of Scalability: Without automated scaling, the system cannot respond dynamically to changes in demand, leading to performance bottlenecks or resource waste.

Inefficient Rollbacks: In the case of a failed deployment, manually rolling back is slow and error-prone.

Poor Observability: Without monitoring integration, diagnosing issues requires digging through logs manually, which slows down incident response.

Onboarding Complexity: New team members face a steep learning curve when deploying or debugging the system manually.

SOLUTIONS

CI/CD Pipeline

- Automate backend deployments using continuous integration and delivery tools (e.g., GitHub Actions, GitLab CI, or Jenkins) to ensure consistency and reduce manual intervention.

Rollback Mechanisms

- Incorporate version tagging and automated rollback steps in the CI/CD process to quickly revert faulty deployments.

Dynamic nginx Configuration Reloads

- Use automation tools or hot-reloading strategies (like `nginx -s reload` or configuration templates) to update routes without downtime.

Infrastructure as Code (IaC)

- Define service configurations (e.g., using Docker Compose) to make deployments reproducible and version-controlled.

SOLUTIONS CONT.

Health Monitoring Stack

- Integrate a monitoring stack (e.g., Prometheus + Grafana + cAdvisor) to track system metrics, visualize performance, and alert on anomalies.

Containerization

- Containerize all services to ensure environment consistency and simplify dependency management across environments.

Auto-Scaling Readiness

- Design the architecture to be compatible with Kubernetes, enabling horizontal or vertical scaling in future iterations.

Service Discovery & Health Checks

- Implement service discovery and health checks (e.g., via nginx, load balancers, or K8s probes) to route traffic only to healthy instances.

KEY BENEFITS

- **Increased Reliability**
 - Automated deployments and dynamic configuration reduce downtime and eliminate human errors during updates.
- **Scalability by Design**
 - The architecture supports integration with Kubernetes, allowing for automatic scaling based on load.
- **Improved Observability**
 - Monitoring dashboards and health checks provide real-time visibility into service status and performance metrics.
- **Reproducible Environments**
 - Containerization and Infrastructure as Code make it easy to reproduce production setups locally or in staging.

TECH STACK APPLICATION SERVICES

Rust App – Main backend logic, handling uploads and processing.

tusd – Resumable upload server implementing the TUS protocol with webhook hooks into the Rust backend.

llamacpp-server – Lightweight local inference server for LLM-based tasks (via [ggml-org/llama.cpp](https://github.com/ggml-org/llama.cpp)).

TECH STACK

NETWORK & PROXY

Nginx (OpenResty) – Acts as a reverse proxy to route external traffic to internal services, with future support for SSL and stub_status for metrics.

TECH STACK MONITORING

Prometheus – Collects system and application metrics from exporters.

Grafana – Dashboards and alerting based on Prometheus metrics.

cAdvisor – Provides detailed container-level resource usage metrics.

nginx-exporter – Exposes Nginx metrics (e.g., requests per second, connections) to Prometheus.

rust-prometheus – Export app metrics to Prometheus.

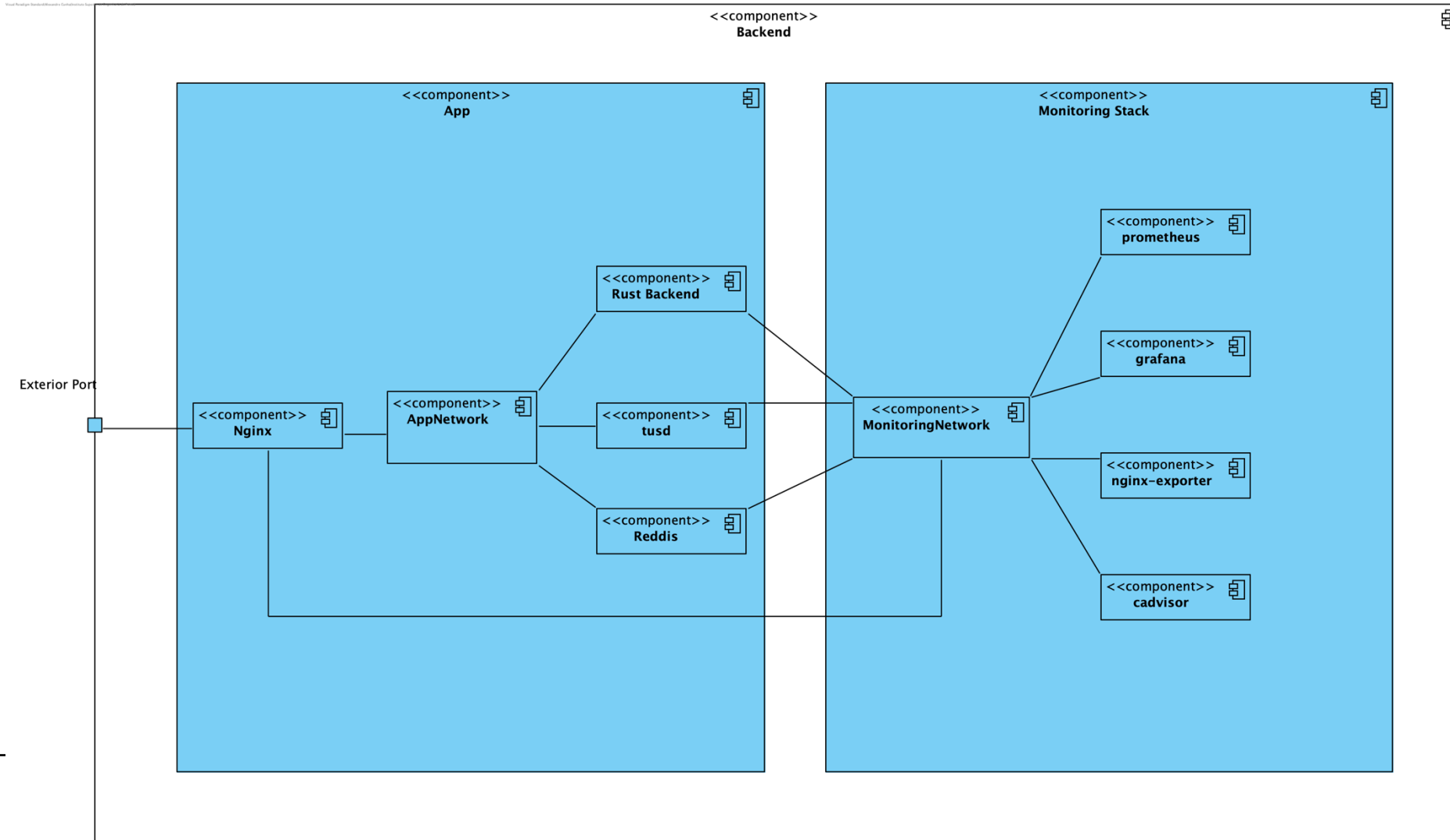
TECH STACK INFRASTRUCTURE

Docker & Docker Compose – Core
toolchain for service definition and
orchestration

Enabling:

- Multi-service architecture
- Inter-service communication via custom networks (tusc-network, mon-net)
- Mounting local directories for live development

ARCHITECTURAL OVERVIEW



APP COMPOSE

The **base docker-compose file** defines and deploys the **main application stack**:

- **Rust App** – Core backend service for upload handling and business logic.
- **tusd** – Handles resumable file uploads; forwards post-finish hooks to the Rust backend.
- **Nginx (OpenResty)** – Acts as reverse proxy; routes requests to both app and tusd services.
- **llamacpp-server** – Optional inference server for lightweight local LLM tasks.
- **Redis** – Supports caching or transient data if needed.

MONITORING STACK COMPOSE

A separate **docker-compose** file is used to deploy the **monitoring stack**, which includes:

- **Prometheus** – Collects metrics from all instrumented containers.
- **Grafana** – Provides real-time dashboards for visualization and alerting.
- **cAdvisor** – Exposes resource usage metrics (CPU, memory, disk I/O) per container.
- **nginx-exporter** – Converts Nginx `stub_status` into Prometheus-friendly metrics.

This separation enables **modularity**: can bring the monitoring stack independently for testing, upgrades, or integration into Kubernetes later.

-  Home
-  Bookmarks
-  Starred
-  Dashboards
-  Explore
-  Drilldown 
-  Alerting
-  Connections
- Add new connection

Data sources

 Administration

 Edit

Export

Share

 Last 3 hours



 Refresh

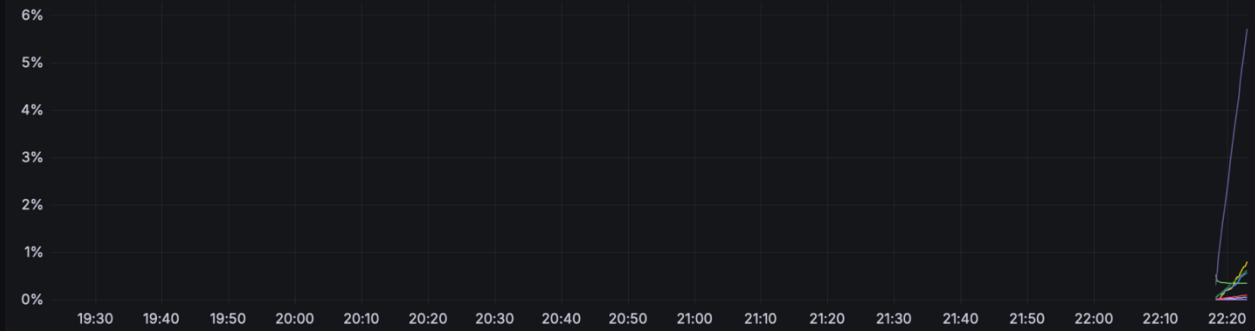
5s

Basic

Running Containers

10

CPU Usage



Name	Max	Mean
cadvisor	5.72%	3.04%
backend-app-1	0.530%	0.362%
redis	0.619%	0.326%
backend-grafana-1	0.799%	0.322%
backend-prometheus-1	0.564%	0.284%
backend-nginx-exporter-1	0.0965%	0.0507%
backend-llamacpp-server-1	0.0456%	0.0250%
backend-nginx-1	0.00598%	0.00312%
backend-tusd-1	0.00230%	0.00136%

Memory

Memory Usage



Name	Max	Mean
cadvisor	4.10 MB	1.87 MB
backend-app-1	2.25 MB	1.07 MB
backend-prometheus-1	1.59 MB	776 kB
backend-llamacpp-server-1	1.24 MB	517 kB
backend-grafana-1	986 kB	253 kB
backend-nginx-exporter-1	273 kB	127 kB
redis	166 kB	81.5 kB
backend-tusd-1	14.7 kB	1.30 kB
backend-nginx-1	94.1 B	66.3 B

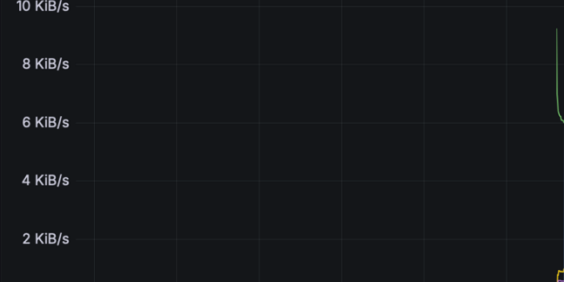
Memory Cached



Name	Max	Mean
backend-app-1	816 kB	557 kB
backend-prometheus-1	2.88 kB	1.69 kB
backend-llamacpp-server-1	3.27 kB	1.17 kB
backend-grafana-1	806 B	433 B
cadvisor	292 B	252 B
backend-nginx-1	0 B	0 B
backend-nginx-exporter-1	0 B	0 B
backend-tusd-1	0 B	0 B
redis	0 B	0 B

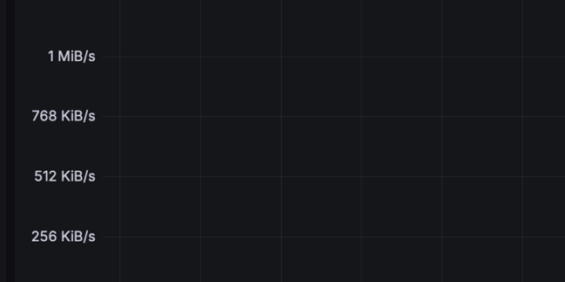
I/O

Reads



Name	Max	Mean
backend-app-1	9.24 KiB/s	6.31 KiB/s
backend-llamacpp-server-1	6.40 KiB/s	2.48 KiB/s
backend-grafana-1	1.44 KiB/s	859 B/s
cadvisor	584 B/s	504 B/s
backend-nginx-1	0 B/s	0 B/s
backend-prometheus-1	0 B/s	0 B/s
backend-tusd-1	0 B/s	0 B/s
redis	0 B/s	0 B/s

Writes



Name	Max	Mean
backend-app-1	1.04 MiB/s	838 KiB/s
backend-prometheus-1	3.51 KiB/s	1.88 KiB/s
backend-grafana-1	945 B/s	214 B/s
backend-llamacpp-server-1	0 B/s	0 B/s
backend-nginx-1	0 B/s	0 B/s
backend-tusd-1	0 B/s	0 B/s
cadvisor	0 B/s	0 B/s
redis	0 B/s	0 B/s

END

Alexandre Cunha, up202400086

Thank you!

Pedro Januário, up202108768
